

Automated Databricks Notebook CICD using GitHub Actions

Automated Databricks Notebook CI/CD using GitHub Actions

Complete Step-by-Step Implementation Guide

Table of Contents

1. Introduction
 2. Architecture Overview
 3. Prerequisites
 4. Environment Setup
 5. GitHub Repository Configuration
 6. Databricks Workspace Configuration
 7. GitHub Actions Workflow Implementation
 8. Testing the CI/CD Pipeline
 9. Data Visualization Integration
 10. Best Practices and Troubleshooting
-

1. Introduction

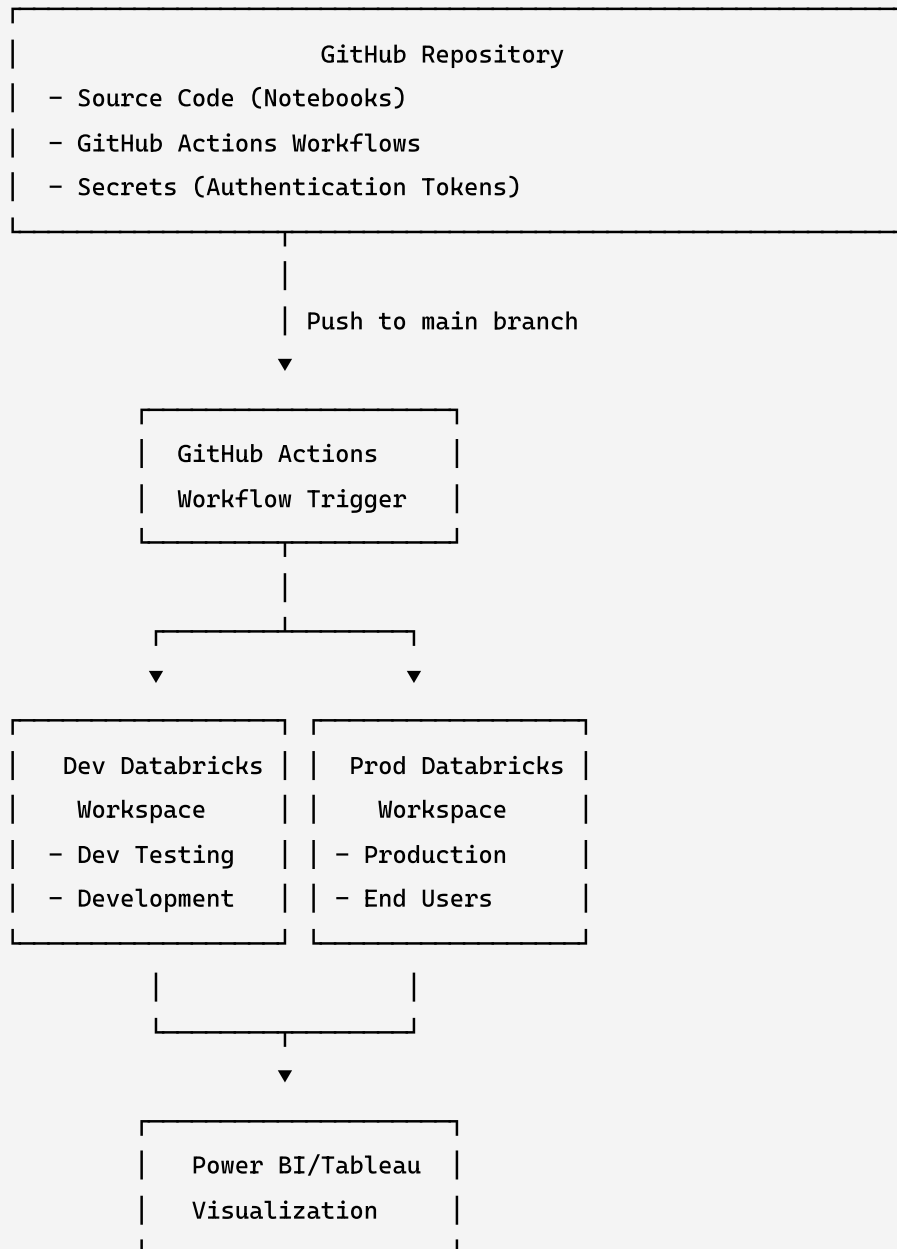
This guide demonstrates how to implement a fully automated Continuous Integration/Continuous Deployment (CI/CD) pipeline for Databricks notebooks using GitHub Actions. This solution enables development teams to maintain separate development and production environments while automating the deployment process through version control.

Key Benefits:

- **Environment Isolation:** Separate dev and prod workspaces with controlled permissions
 - **Automated Deployments:** Changes automatically flow from development to production
 - **Version Control:** Full Git history of all notebook changes
 - **Quality Assurance:** Pull request workflow ensures code review before production deployment
 - **Rollback Capability:** Easy reversion to previous versions if issues arise
-

2. Architecture Overview

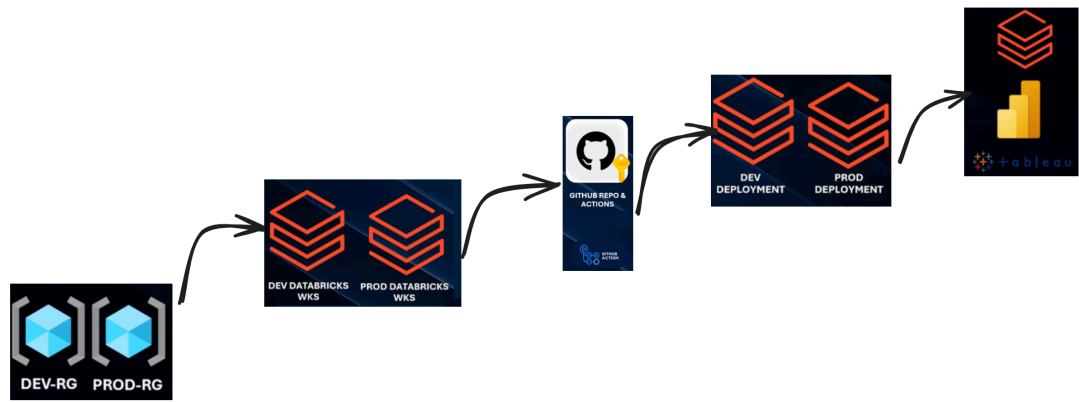
The solution architecture consists of the following components:



Workflow Process:

1. Developers create feature branches and commit changes
2. Pull requests are created to merge into the main branch
3. Upon merge, GitHub Actions automatically deploys to dev workspace
4. After successful dev deployment, changes are deployed to prod workspace
5. Production data is available for visualization tools

Automated Databricks Notebook CI/CD using Git Actions



3. Prerequisites

Before beginning implementation, ensure you have:

Required Access:

- Azure subscription with permissions to create resource groups
- GitHub account with repository creation rights
- Databricks workspace creation permissions
- Power BI Desktop (for visualization)

Required Knowledge:

- Basic understanding of Git workflows
- Familiarity with Azure portal navigation
- Basic PySpark/Python knowledge
- Understanding of CI/CD concepts

4. Environment Setup

4.1 Create Azure Resource Groups

Resource groups provide logical containers for Azure resources and enable environment isolation.

Step 1: Navigate to Azure Portal

- Log in to [Azure Portal](#)
- Click on "Resource groups" from the left navigation menu
- Click "+ Create" button

Step 2: Create Development Resource Group

Name: dev-databricks-rg

Region: East US (or your preferred region)

- Click "Review + create"
- Click "Create"

Step 3: Create Production Resource Group

Name: prod-databricks-rg

Region: East US (same as dev for consistency)

- Click "Review + create"
- Click "Create"

Best Practice: Use consistent naming conventions across environments. The pattern `{environment}-{service}-rg` helps identify resources quickly.

4.2 Provision Databricks Workspaces

Databricks workspaces provide isolated environments for data engineering and analytics.

Step 1: Create Development Workspace

1. From Azure Portal home, click "Create a resource"
2. Search for "Azure Databricks"
3. Click "Create"
4. Configure the workspace:

Subscription: [Your subscription]

Resource Group: dev-databricks-rg

Workspace Name: dev-databricks-workspace

Region: East US

Pricing Tier: Premium (for RBAC support)

5. Click "Review + create" → "Create"
6. Wait for deployment (approximately 3-5 minutes)

Step 2: Create Production Workspace

1. Repeat the process with these settings:

```
Resource Group: prod-databricks-rg
Workspace Name: prod-databricks-workspace
Region: East US
Pricing Tier: Premium
```

2. Click "Review + create" → "Create"

Why Separate Workspaces? This isolation prevents development activities from impacting production workloads and enables granular permission control.

5. GitHub Repository Configuration

5.1 Create GitHub Repository

Step 1: Create New Repository

1. Navigate to [GitHub](#)
2. Click the "+" icon → "New repository"
3. Configure repository:

```
Repository name: cicd-databricks-project
Description: Automated Databricks CI/CD implementation
Visibility: Public (or Private based on requirements)
☒ Initialize with README
```

4. Click "Create repository"

5.2 Configure GitHub Personal Access Token

Personal Access Tokens (PAT) provide secure authentication between Databricks and GitHub.

Step 1: Generate PAT

1. Click your profile picture → "Settings"
2. Scroll to "Developer settings" (bottom left)
3. Click "Personal access tokens" → "Tokens (classic)"
4. Click "Generate new token" → "Generate new token (classic)"
5. Configure token:

Note: databricks-tokenExpiration: 90 days

Scopes:

- ☒ repo (all repo permissions)
- ☒ workflow (for GitHub Actions)

6. Click "Generate token"
7. **IMPORTANT:** Copy the token immediately (it won't be shown again)

Security Note: Store tokens securely. Never commit tokens to repositories or share them in plain text.

5.3 Configure Repository Secrets

GitHub Secrets store sensitive information securely for use in workflows.

Step 1: Navigate to Repository Settings

1. Go to your repository
2. Click "Settings" tab
3. Navigate to "Secrets and variables" → "Actions"
4. Click "New repository secret"

Step 2: Create Required Secrets

Create four secrets with the following information:

Secret 1: DEV_DEPLOYMENT_TARGET_URL

Name: DEV_DEPLOYMENT_TARGET_URL
Value: [Your dev workspace URL without https://]
Example: adb-1954722681802715.15.azuredatabricks.net/

How to get the URL:

- Open dev Databricks workspace
- Copy URL from browser address bar
- Remove `https://` prefix
- Remove any trailing paths (keep only the domain)

Secret 2: DEV_DEPLOYMENT_TARGET_TOKEN

Name: DEV_DEPLOYMENT_TARGET_TOKEN

dapi797ba3d57e3015994efc68aa88307841-3
Value: [Access token from dev workspace]

How to generate token:

1. Open dev Databricks workspace
2. Click user icon (top right) → "Settings"
3. Navigate to "Developer" tab

4. Click "Manage" next to "Access tokens"
5. Click "Generate new token"

```
Comment: dev-tokenLifetime: 90 days
```

6. Click "Generate"
7. Copy token immediately

Secret 3: PROD_DEPLOYMENT_TARGET_URL

```
Name: PROD_DEPLOYMENT_TARGET_URL  
Value: [Your prod workspace URL without https://]
```

Secret 4: PROD_DEPLOYMENT_TARGET_TOKEN

```
Name: PROD_DEPLOYMENT_TARGET_TOKEN  
Value: [Access token from prod workspace]  
dapi8eb00e29d9c93485ad4fa4459ba79363-3
```

Verification: After creating all secrets, you should see four secrets listed in the Actions secrets page.

6. Databricks Workspace Configuration

6.1 Link GitHub Repository to Databricks

This step connects your Databricks workspaces to the GitHub repository.

Step 1: Configure Dev Workspace

1. Open dev Databricks workspace
2. Click "Workspace" in left navigation
3. Click your user folder
4. Click "Create" → "Git folder"
5. Configure Git folder:

```
Git repository URL: https://github.com/[username]/cicd-databricks-project  
Git provider: GitHub (auto-detected)Repository  
name: dev-databricks-git-folder
```

6. Click "Create Git folder"

Step 2: Configure Prod Workspace

1. Open prod Databricks workspace
2. Repeat steps 2-5 with:

Repository name: prod-databricks-git-folder

Naming Convention: Using different folder names (dev/prod prefix) helps distinguish environments at a glance.

6.2 Authenticate GitHub in Databricks

Step 1: Configure Dev Workspace Authentication

1. In dev workspace, click user icon → "Settings"
2. Navigate to "Linked accounts"
3. Select "GitHub" as Git provider
4. Choose "Personal access token"
5. Enter credentials:

Username: [Your GitHub username]

Personal Access Token: [Token created in section 5.2]

6. Click "Save"
7. Verify "Credential saved" message appears

Step 2: Configure Prod Workspace Authentication

1. Repeat steps in prod workspace
2. Use the same GitHub username and token

Note: You can reuse the same personal access token for both workspaces.

6.3 Get Repository Paths

You'll need the full repository paths for the GitHub Actions workflow.

Step 1: Get Dev Repository Path

1. In dev workspace, navigate to Workspace
2. Expand your user folder
3. Right-click "dev-databricks-git-folder"
4. Click "Copy path"
5. Save this path (example format):


```
/Workspace/Users/admin@nikhil0073outlook.onmicrosoft.com/dev-databricks-git-folder
```

```
/Workspace/Users/admin@nikhil0073outlook.onmicrosoft.com/prod-databricks-git-folder
```

```
/Workspace/Users/your.email@domain.com/dev-databricks-git-folder
```

Step 2: Get Prod Repository Path

1. Repeat in prod workspace for "prod-databricks-git-folder"
2. Save this path

Important: These paths will be used in the GitHub Actions workflow YAML file.

7. GitHub Actions Workflow Implementation

7.1 Understanding the Workflow Structure

The workflow file defines the automated deployment process. Let's break down each component:

Workflow Components:

- **Trigger:** Defines when the workflow runs
- **Jobs:** Separate deployment tasks (dev and prod)
- **Steps:** Individual actions within each job
- **Environment Variables:** Store configuration values

7.2 Create Workflow File

Step 1: Navigate to Actions

1. Go to your GitHub repository
2. Click "Actions" tab
3. Click "Set up a workflow yourself"

Step 2: Create main.yml

The workflow will be automatically created at: `.github/workflows/main.yml`

Step 3: Add Workflow Configuration

```

name: main

on:
  push:
    paths-ignore:
      - .github/**
    branches:
      - main

jobs:
  deploy-dev:
    runs-on: ubuntu-latest
    environment: development
    env:
      DATABRICKS_HOST: https://${{ secrets.DEV_DEPLOYMENT_TARGET_URL }}
      DATABRICKS_TOKEN: ${{ secrets.DEV_DEPLOYMENT_TARGET_TOKEN }}
      REPO_PATH: /Workspace/Users/your.email@domain.com/dev-databricks-git-folder

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v3
        with:
          python-version: 3.8

      - name: Set up Databricks CLI
        uses: databricks/setup-cli@main

      - name: Extract branch name
        id: extract_branch
        run: echo "branch=$(echo ${GITHUB_REF#refs/heads/})" >> $GITHUB_OUTPUT

      - name: Update Databricks Repo (Dev)
        run: |
          databricks repos update "${{ env.REPO_PATH }}" \
            --branch "${{ steps.extract_branch.outputs.branch }}"

  deploy-prod:
    needs: deploy-dev
    runs-on: ubuntu-latest
    environment: production
    env:
      DATABRICKS_HOST: https://${{ secrets.PROD_DEPLOYMENT_TARGET_URL }}

```

```

DATABRICKS_TOKEN: ${ secrets.PROD_DEPLOYMENT_TARGET_TOKEN }
REPO_PATH: /Workspace/Users/your.email@domain.com/prod-databricks-git-folder

steps:
  - name: Checkout repository
    uses: actions/checkout@v3

  - name: Set up Python
    uses: actions/setup-python@v3
    with:
      python-version: 3.8

  - name: Set up Databricks CLI
    uses: databricks/setup-cli@main

  - name: Extract branch name
    id: extract_branch_prod
    run: echo "branch=$(echo ${GITHUB_REF#refs/heads/})" >> $GITHUB_OUTPUT

  - name: Update Databricks Repo (Prod)
    run: |
      databricks repos update "${ env.REPO_PATH }" \
        --branch "${ steps.extract_branch_prod.outputs.branch }"

```

Step 4: Update Repository Paths

- Replace the `REPO_PATH` values in both jobs with your actual paths from section 6.3

Step 5: Commit Workflow

1. Click "Commit changes"
2. Add commit message: "Add CI/CD workflow configuration"
3. Select "Commit directly to the main branch"
4. Click "Commit changes"

7.3 Workflow Configuration Explained

Let's understand each section of the workflow:

Trigger Configuration:

YAML

```
on:
  push:
    paths-ignore:
      - .github/**
    branches:
      - main
```

- **on: push:** Workflow runs when code is pushed
- **paths-ignore:** Prevents workflow from triggering on changes to workflow files themselves
- **branches: - main:** Only triggers on pushes to the main branch

Job Dependencies:

YAML

```
deploy-prod:
  needs: deploy-dev
```

- Production deployment only starts after dev deployment succeeds
- Ensures changes are tested in dev before reaching production

Environment Variables:

YAML

```
env:
  DATABRICKS_HOST: https://${{ secrets.DEV_DEPLOYMENT_TARGET_URL }}
  DATABRICKS_TOKEN: ${{ secrets.DEV_DEPLOYMENT_TARGET_TOKEN }}
  REPO_PATH: /Workspace/Users/ ...
```

- Securely accesses secrets without exposing values
- **https://** prefix is added to the URL secret
- REPO_PATH must match your actual Databricks workspace path

Key Steps Explained:

1. **Checkout repository:** Downloads repository code to the runner
2. **Set up Python:** Installs Python 3.8 for Databricks CLI
3. **Set up Databricks CLI:** Installs the Databricks command-line tool
4. **Extract branch name:** Gets the current Git branch name
5. **Update Databricks Repo:** Syncs the Databricks Git folder with GitHub

8. Testing the CI/CD Pipeline

8.1 Create Development Notebook

Step 1: Create Feature Branch in Dev Workspace

1. Open dev Databricks workspace
2. Navigate to Workspace → dev-databricks-git-folder
3. Click current branch (main)
4. Click "Create branch"
5. Name: `feature/create-databricks-notebook`
6. Base: main
7. Click "Create"

Step 2: Create Notebook

1. Click "Create" → "Notebook"
2. Name: `sales_data_notebook_for_data_processing`
3. Press Enter

Step 3: Add PySpark Code

Add the following code cells to your notebook:

Cell 1: Schema Definition and Data Creation

PYTHON

```

from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, DateType
from pyspark.sql import Row
from datetime import datetime

# Define schema
sales_schema = StructType([
    StructField("OrderID", StringType(), False),
    StructField("OrderDate", DateType(), False),
    StructField("Region", StringType(), True),
    StructField("Product", StringType(), False),
    StructField("Category", StringType(), False),
    StructField("Price", DoubleType(), False),
    StructField("Quantity", IntegerType(), False)
])

# Sample data
sales_data = [
    ("ORD009", datetime(2022, 1, 10), "West", "iPhone 15", "Electronics", 1099.99,
3),
    ("ORD010", datetime(2022, 2, 18), "East", "MacBook Pro", "Electronics",
1999.99, 1),
    ("ORD011", datetime(2022, 3, 6), "South", "Ergonomic Chair", "Furniture",
179.99, 2),
    ("ORD012", datetime(2022, 4, 20), "North", "4K Monitor", "Electronics", 329.99,
2),
    ("ORD013", datetime(2022, 5, 27), "West", "Standing Desk", "Furniture", 499.99,
1),
    ("ORD014", datetime(2022, 6, 15), "East", "iPad Pro", "Electronics", 899.99,
2),
    ("ORD015", datetime(2022, 7, 23), "South", "Corner Bookshelf", "Furniture",
249.99, 3),
    ("ORD016", datetime(2022, 8, 12), "North", "Laser Printer", "Electronics",
199.99, 1),
    ("ORD001", datetime(2023, 1, 15), "West", "iPhone 14", "Electronics", 999.99,
2),
    ("ORD002", datetime(2023, 2, 20), "East", "MacBook Air", "Electronics",
1199.49, 1),
    ("ORD003", datetime(2023, 3, 5), "South", "Desk Chair", "Furniture", 149.99,
4),
    ("ORD004", datetime(2023, 4, 17), "North", "Monitor", "Electronics", 249.99,
2),
    ("ORD005", datetime(2023, 5, 30), "West", "Office Desk", "Furniture", 399.99,
1),
    ("ORD006", datetime(2023, 6, 12), "East", "iPad", "Electronics", 499.99, 3),

```

```

("ORD007", datetime(2023, 7, 25), "South", "Bookshelf", "Furniture", 199.99,
2),
("ORD008", datetime(2023, 8, 8), "North", "Printer", "Electronics", 149.49, 1),
]

# Create DataFrame
sales_df = spark.createDataFrame(data=sales_data, schema=sales_schema)
sales_df.display()

```

Cell 2: Data Transformation

PYTHON

```

# Data Transformation
from pyspark.sql.functions import year, month, col, expr

sales_transformed = (
    sales_df.withColumn("Year", year(col("OrderDate")))
             .withColumn("Month", month(col("OrderDate")))
             .withColumn("SalesAmount", expr("Price * Quantity"))
)

sales_transformed.display()

```

Cell 3: Aggregation

PYTHON

```

# Aggregation
sales_summary = (
    sales_transformed.groupBy("Region", "Category")
                     .sum("SalesAmount")
                     .withColumnRenamed("sum(SalesAmount)", "TotalSales")
                     .orderBy("Region", "Category")
)

sales_summary.display()

```

Understanding the Code:

- **Schema Definition:** Defines data types for each column ensuring data quality
- **Data Transformation:** Adds Year, Month, and calculated SalesAmount columns
- **Aggregation:** Groups data by Region and Category to calculate total sales

8.2 Commit and Create Pull Request

Step 1: Commit Changes

1. Navigate back to Workspace → dev-databricks-git-folder

2. Click on feature branch
3. Notice the "+" indicator (uncommitted changes)
4. Add commit message: `Created sales data notebook`
5. Click "Commit and push"
6. Confirmation message appears
7. Click "Create pull request"

Step 2: Complete Pull Request

1. GitHub opens in new tab
2. Review changes (title and description auto-populated)
3. Click "Create pull request"
4. System checks for conflicts
5. Click "Merge pull request"
6. Click "Confirm merge"
7. Pull request status: "Merged and closed"
8. Click "Delete branch" (cleanup)

Step 3: Verify in Main Branch

1. Go to repository main branch
2. Navigate to notebook file: `sales_data_notebook_for_data_processing.ipynb`
3. Verify code is present

Important: At this point, the notebook exists in GitHub and dev workspace, but NOT in prod workspace. This is intentional and demonstrates environment isolation.

8.3 Verify Initial Deployment

Check Dev Workspace:

1. Open dev Databricks workspace
2. Navigate to dev-databricks-git-folder
3. Switch to main branch
4. Verify notebook appears

Check Prod Workspace:

1. Open prod Databricks workspace
2. Navigate to prod-databricks-git-folder
3. Notice notebook is NOT present
4. This confirms production isolation

Why isn't the notebook in prod? The GitHub Actions workflow hasn't been triggered yet because we committed directly to main through the merge, not through a subsequent push.

8.4 Add Delta Table Save Functionality

Now we'll add code to save data as a Delta table, which will trigger the CI/CD pipeline.

Step 1: Create New Feature Branch

1. In dev workspace, navigate to dev-databricks-git-folder
2. Switch to main branch
3. Click branch dropdown → "Create branch"
4. Name: `feature/save-as-table`
5. Base: main
6. Click "Create"

Step 2: Add Save Code

1. Open `sales_data_notebook_for_data_processing`
2. Scroll to bottom
3. Click "+ Code" to add new cell
4. Add this code:

```
PYTHON
# Save as Delta Table (overwrite mode)
sales_transformed.write.format("delta").mode("overwrite").saveAsTable("llm.sales_sc
hema.sales_gitt_table")
```

Note: We'll create the Unity Catalog, schema, and table in production before running this code.

Step 3: Commit and Merge

1. Navigate to git folder
2. Select feature branch
3. Add commit message: `Added save as table functionality`
4. Click "Commit and push"
5. Click "Create pull request"
6. In GitHub, click "Create pull request"
7. Click "Merge pull request"
8. Click "Confirm merge"
9. Delete feature branch

8.5 Monitor GitHub Actions

Step 1: Navigate to Actions

1. Go to GitHub repository
2. Click "Actions" tab
3. You'll see the workflow running
4. Click on the workflow run

Step 2: Monitor Deployment Jobs

You'll see two jobs:

- **deploy-dev:** Shows deployment to development environment
- **deploy-prod:** Shows deployment to production (waits for dev to complete)

Step 3: View Job Details

1. Click "deploy-dev"
2. Expand each step to view logs:
 - ✓ Set up job
 - ✓ Checkout repository
 - ✓ Set up Python
 - ✓ Set up Databricks CLI
 - ✓ Extract branch name
 - ✓ Update Databricks Repo (Dev)
 - ✓ Complete job
3. Click "deploy-prod"
4. Verify similar successful steps

Success Indicators:

- Green checkmarks next to all steps
- "deploy-dev" and "deploy-prod" both show success
- Total workflow time typically 2-3 minutes

8.6 Verify Production Deployment

Step 1: Check Prod Workspace

1. Open prod Databricks workspace
2. Refresh browser if already open
3. Navigate to prod-databricks-git-folder
4. Switch to main branch (if needed)
5. **Verify:** `sales_data_notebook_for_data_processing` now appears

Step 2: Review Notebook Code

1. Open the notebook
2. Scroll through all cells
3. Verify all code matches dev version
4. Confirm save table code is present

Achievement Unlocked: Your CI/CD pipeline is now fully functional! Changes flow automatically from dev to prod.

9. Data Visualization Integration

9.1 Create Unity Catalog Structure

Before running the notebook in production, create the required Unity Catalog structure.

Step 1: Create Catalog

1. In prod workspace, click "Catalog" (left navigation)
2. Click "+ Add" → "Add catalog"
3. Configure:

```
Name: one_data_uc Storage location: Managed by Databricks
```

4. Click "Create"

Step 2: Create Schema

1. Click on "one_data_uc" catalog
2. Click "+ Add" → "Add schema"
3. Configure:

```
Name: sales_schema Storage location: Managed by Databricks
```

4. Click "Create"

Understanding Unity Catalog:

- **Catalog:** Top-level container (like a database in traditional systems)
- **Schema:** Logical grouping of tables within a catalog
- **Table:** Actual data storage

Hierarchy:

```
one_data_uc (Catalog)
├── sales_schema (Schema)
│   └── sales_gitt_table (Table - will be created by notebook)
```

9.2 Execute Notebook in Production

Step 1: Run All Cells

1. Open `sales_data_notebook_for_data_processing` in prod workspace
2. Click "Run all" at top
3. Notebook connects to serverless compute
4. Wait for all cells to execute (approximately 1-2 minutes)

Step 2: Verify Execution

Each cell should show:

- Green checkmark ✓ indicating success
- Execution time displayed
- Output data displayed (for `display()` commands)

Step 3: Verify Table Creation

1. Navigate to Catalog
2. Expand "one_data_uc"
3. Expand "sales_schema"
4. Verify "sales_gitt_table" exists
5. Click table name
6. Click "Sample data" tab
7. Verify records for 2022 and 2023 appear

9.3 Handle Notebook Modifications

Important Consideration: Running notebook cells creates modifications in the Git folder.

Step 1: Commit Changes

1. Navigate to prod-databricks-git-folder
2. Notice modification indicator on main branch
3. Click branch dropdown
4. Add commit message: `Executed all cells`
5. Click "Commit and push"
6. Click "Create pull request"

Step 2: Complete Pull Request

- In GitHub, you'll see "nothing to compare" if no actual code changed
- This is expected (only execution metadata changed)
- The action will still run to maintain consistency

Step 3: Monitor Action

1. Go to Actions tab
2. Verify workflow runs successfully
3. Both deploy-dev and deploy-prod should succeed

Best Practice: In production environments, use scheduled jobs instead of manual cell execution to avoid frequent commits.

9.4 Configure Power BI Connection

Step 1: Get Connection File

1. In prod workspace, navigate to Catalog
2. Click "one_data_uc" → "sales_schema" → "sales_gitt_table"
3. Click "Open in" dropdown
4. Select "Power BI Desktop"
5. Click "Download connection file"
6. File downloads to your computer

Step 2: Generate Access Token

1. In prod workspace, click user icon → "Settings"
2. Go to "Developer" tab
3. Click "Manage" (Access tokens section)
4. Click "Generate new token"

Comment: Power BI tokenLifetime (days): 90

5. Click "Generate"
6. Copy token immediately
7. Click "Done"

Step 3: Open in Power BI

1. Locate downloaded connection file
2. Double-click to open in Power BI Desktop
3. Connection dialog appears
4. Select "Personal Access Token"
5. Paste token in authentication field
6. Click "Connect"

Step 4: Load Data

1. Table appears in navigator
2. Click table name to preview
3. Click "Load" to import data

9.5 Create Visualizations

Step 1: Create Basic Report

1. In Power BI, expand table in Fields pane
2. Drag "SalesAmount" to canvas
3. Visual shows total: approximately 35,000-46,000

Step 2: Add Year Dimension

1. Drag "Year" field to canvas
2. Click dropdown on "Year" in visual
3. Select "Don't summarize"
4. Now Year shows as dimension (2022, 2023) not aggregated

Step 3: Create Table Visual

1. Select "Table" visualization
2. Drag "Year" to Columns
3. Drag "SalesAmount" to Values
4. Table shows:

Year	SalesAmount2022	[amount]2023	[amount]
------	-----------------	--------------	----------

Current State: Reports show only 2022 and 2023 data, which matches our current dataset.

9.6 Test with Additional Data

Let's add 2024 data to verify the pipeline handles data additions.

Step 1: Create Feature Branch for 2024 Data

1. In dev workspace, go to dev-databricks-git-folder
2. Switch to main branch
3. Create branch: `feature/sales-2024-data`
4. Base: main

Step 2: Add 2024 Data

1. Open notebook
2. Find the sales_data list
3. Add these rows inside the list:

PYTHON

```
(
"ORD009", datetime(2024, 1, 10), "West", "iPhone 15", "Electronics", 1099.99, 3),
"ORD010", datetime(2024, 2, 18), "East", "MacBook Pro", "Electronics", 1999.99,
1),
"ORD011", datetime(2024, 3, 6), "South", "Ergonomic Chair", "Furniture", 179.99,
2),
"ORD012", datetime(2024, 4, 20), "North", "4K Monitor", "Electronics", 329.99, 2),
"ORD013", datetime(2024, 5, 27), "West", "Standing Desk", "Furniture", 499.99, 1),
"ORD014", datetime(2024, 6, 15), "East", "iPad Pro", "Electronics", 899.99, 2),
"ORD015", datetime(2024, 7, 23), "South", "Corner Bookshelf", "Furniture", 249.99,
3),
"ORD016", datetime(2024, 8, 12), "North", "Laser Printer", "Electronics", 199.99,
1),
```

Step 3: Deploy Changes

1. Commit: **Added sales 2024 data**
2. Push to feature branch
3. Create pull request
4. Merge to main
5. Delete feature branch

Step 4: Monitor Deployment

1. Go to Actions tab
2. Wait for both jobs to complete (green checkmarks)
3. Verify deployment timestamps

Step 5: Run Production Notebook

1. Open prod workspace
2. Refresh if needed
3. Open notebook (verify 2024 data present in code)
4. Click "Run all"
5. Wait for execution

Step 6: Verify in Catalog

1. Navigate to Catalog
2. Open table: one_data_uc.sales_schema.sales_gitt_table
3. Go to "Sample data" tab
4. Scroll down to verify 2024 records appear

Step 7: Refresh Power BI

1. In Power BI Desktop, click "Refresh" (top ribbon)
2. Wait for refresh to complete

3. Verify 2024 now appears in table visual
4. Total sales amount updates to include 2024

Step 8: Final Test - Add 2025 Data

Repeat the process for 2025 data:

1. Create branch: `feature/sales-2025-data`
2. Add data:

```
PYTHON
("ORD017", datetime(2025, 1, 10), "West", "iPhone 15", "Electronics", 1099.99, 3),
("ORD018", datetime(2025, 2, 18), "East", "MacBook Pro", "Electronics", 1999.99,
1),
("ORD019", datetime(2025, 3, 6), "South", "Ergonomic Chair", "Furniture", 179.99,
2),
("ORD020", datetime(2025, 4, 20), "North", "4K Monitor", "Electronics", 329.99, 2),
("ORD021", datetime(2025, 5, 27), "West", "Standing Desk", "Furniture", 499.99, 1),
("ORD022", datetime(2025, 6, 15), "East", "iPad Pro", "Electronics", 899.99, 2),
("ORD023", datetime(2025, 7, 23), "South", "Corner Bookshelf", "Furniture", 249.99,
3),
("ORD024", datetime(2025, 8, 12), "North", "Laser Printer", "Electronics", 199.99,
1),
```

3. Follow deployment process
4. Verify in Power BI

Final Verification: Power BI should now show:

Year	SalesAmount
2022	[amount]
2023	[amount]
2024	[amount]
2025	[amount]
Total:	~35,000-46,000

10. Best Practices and Troubleshooting

10.1 Best Practices

Development Workflow:

1. **Always use feature branches** for development
 - Never commit directly to main in dev workspace

- Use descriptive branch names (e.g., `feature/add-customer-data`)

2. Code Review Process

- Use pull requests for all changes
- Add meaningful commit messages
- Review code before merging

3. Testing Strategy

- Test notebooks thoroughly in dev before merging
- Verify all cells execute successfully
- Check data quality and transformations

Production Management:

1. Avoid Manual Changes

- Never modify code directly in prod workspace
- All changes should flow through CI/CD
- Document any emergency changes

2. Scheduling vs Manual Execution

- Use scheduled jobs in production
- Avoid running cells manually (causes Git modifications)
- Configure appropriate retry policies

3. Monitoring

- Regularly check GitHub Actions for failures
- Set up notifications for workflow failures
- Monitor Databricks job runs

Security:

1. Token Management

- Rotate access tokens regularly (every 90 days)
- Use minimal required permissions
- Store tokens in GitHub Secrets only
- Never commit tokens to repository

2. Access Control

- Limit production workspace access
- Use separate service accounts for automation
- Implement proper RBAC in Databricks

10.2 Common Issues and Solutions

Issue 1: Workflow Fails on Deploy Step

```
Error: Databricks authentication failed
```

Solution:

- Verify secrets are correctly set in GitHub
- Check token hasn't expired
- Ensure `https://` is prepended to URL in workflow
- Verify repository path is correct

Issue 2: Notebook Not Appearing in Production Symptoms:

- Dev deployment succeeds
- Prod deployment succeeds
- Notebook still missing in prod workspace

Solution:

1. Refresh prod workspace browser
2. Verify branch is set to main
3. Check GitHub Actions logs for actual errors
4. Verify REPO_PATH in workflow matches prod path exactly

Issue 3: Git Modifications Keep Appearing Symptoms:

- Constant modification indicators
- Frequent pull requests needed

Solution:

- Stop running cells manually in prod
- Use scheduled jobs instead
- If needed, commit executions with meaningful messages

Issue 4: Power BI Connection Fails

```
Error: Authentication failed
```

Solution:

- Regenerate personal access token
- Verify token has sufficient permissions
- Check network connectivity to Databricks
- Try downloading connection file again

Issue 5: Merge Conflicts Symptoms:

- Pull request shows conflicts
- Unable to merge automatically

Solution:

1. In Databricks workspace, pull latest main
2. Switch to feature branch
3. Resolve conflicts in notebook
4. Commit resolution
5. Retry merge

10.3 Workflow Debugging

Enable Debug Logging: Add this to workflow for detailed logs:

```
env:
  ACTIONS_RUNNER_DEBUG: true
  ACTIONS_STEP_DEBUG: true
```

YAML**Check Deployment Status:**

1. Navigate to Actions tab
2. Click failing workflow
3. Expand failed step
4. Review error messages
5. Check timestamps for timeout issues

Verify Databricks CLI: Test CLI commands manually:

```
databricks repos list
databricks repos update /path/to/repo --branch main
```

SHELL

10.4 Performance Optimization

Workflow Optimization:

- Use cached dependencies where possible
- Parallel job execution for independent tasks
- Optimize Python environment setup

Databricks Optimization:

- Use appropriate cluster sizes
- Enable auto-scaling for variable workloads
- Leverage Delta Lake optimizations
- Configure proper retry policies

10.5 Disaster Recovery

Rollback Strategy:

1. **Identify Problem:** Determine which commit caused issues
2. **Revert in GitHub:**

```
git revert <commit-hash>git push
```

SHELL

3. **Monitor Deployment:** Watch GitHub Actions redeploy previous version
4. **Verify Production:** Check prod workspace updated correctly

Backup Strategy:

- Maintain separate backup catalog
- Export critical notebooks periodically
- Document manual recovery procedures
- Test recovery process quarterly

10.6 Scaling Considerations

Multiple Environments:

- Add staging environment between dev and prod
- Modify workflow to include staging job
- Create separate secrets for each environment

Multiple Teams:

- Use separate Git folders per team
- Implement code owners for approval
- Create team-specific workflows
- Use environment protection rules

Multiple Regions:

- Deploy to multiple Databricks regions
- Configure geo-specific secrets
- Implement region-specific workflows
- Consider data residency requirements

Conclusion

You've successfully implemented a production-ready CI/CD pipeline for Databricks notebooks using GitHub Actions. This solution provides:

✓ **Automated Deployments:** Changes flow automatically from dev to prod ✓ **Environment Isolation:** Separate workspaces prevent dev/prod conflicts ✓ **Version Control:** Full Git history of all changes ✓ **Quality Gates:** Pull request workflow ensures review ✓ **Reproducibility:** Consistent deployment process every time

Key Takeaways:

1. Feature branch workflow promotes safe development
2. GitHub Actions provides reliable automation
3. Environment separation is crucial for production stability
4. Proper authentication and secrets management ensures security
5. Integration with visualization tools completes the data pipeline

Next Steps:

- Implement additional environments (staging, UAT)
- Add automated testing to workflows
- Set up monitoring and alerting
- Create documentation for your team
- Explore advanced Databricks features (Jobs, MLflow)

This architecture is enterprise-ready and can be adapted for organizations of any size. The principles demonstrated here apply to any CI/CD implementation, not just Databricks.

Additional Resources

Databricks Documentation:

- [Databricks Repos](#)
- [Unity Catalog](#)
- [Databricks CLI](#)

GitHub Documentation:

- [GitHub Actions](#)
- [Encrypted Secrets](#)
- [Workflow Syntax](#)

Best Practices:

- [Git Branching Strategies](#)
- [CI/CD Patterns](#)
- [DevOps Principles](#)